FIL GRADUATE AI COHORT · HANDS-ON LAB

Make RAG Better

Improve retrieval and generation with Multi-Query Retrieval (MQR) and Reciprocal Rank Fusion (RRF)



FastAPI · LangChain · FAISS · React · azure/gpt-6-luna · ada-002

Today's journey: five steps, one chatbot

1

See the problem

Basic RAG misses chunks. We watch it fail.

›

2

MQR

LLM rewrites the question. More recall.

›

3

RRF

Rank twice, fuse. Fewer, better chunks.

›

4

Grounding

Numbered passages, citations, budget.

›

5

Measure

Hit rate, MRR, chunks sent.

Code map Part A: app/mqr.py (TODO 1–4) · Part B: app/rerank.py (TODO 5–8) · Part C: app/generation.py (TODO 9–10)

Recap: Basic RAG is already in your starter



Offline, once at startup

- 12 handbook files → split by Markdown heading → 59 chunks
- Each chunk: "<doc title> > <section>" + text, id like 04_payment_error_codes.md#2
- Embedded in batches, stored in FAISS IndexFlatIP (unit vectors, so inner product = cosine)

In code (app/basic_rag.py)

```
chain = RunnableParallel(
    context=retriever,      # FAISS top-5
    question=RunnablePassthrough(),
).assign(answer=
    prompt | gateway_llm | StrOutputParser()
)
```

Three ways Basic RAG fails on real questions

Vocabulary mismatch

"Can I WFH on Fridays?"

Handbook says: hybrid model, anchor days, home location

Fix: MQR rewrites

Exact identifiers

"PAY-E4035 keeps coming in prod"

6 sibling codes look the same to an embedding model

Fix: BM25 inside RRF

Vague questions

"deploy on 30th?"

Answer lives under 'change freeze, 28th to 2nd'

Fix: MQR + RRF

If the right chunk never reaches the prompt, no LLM can answer correctly.

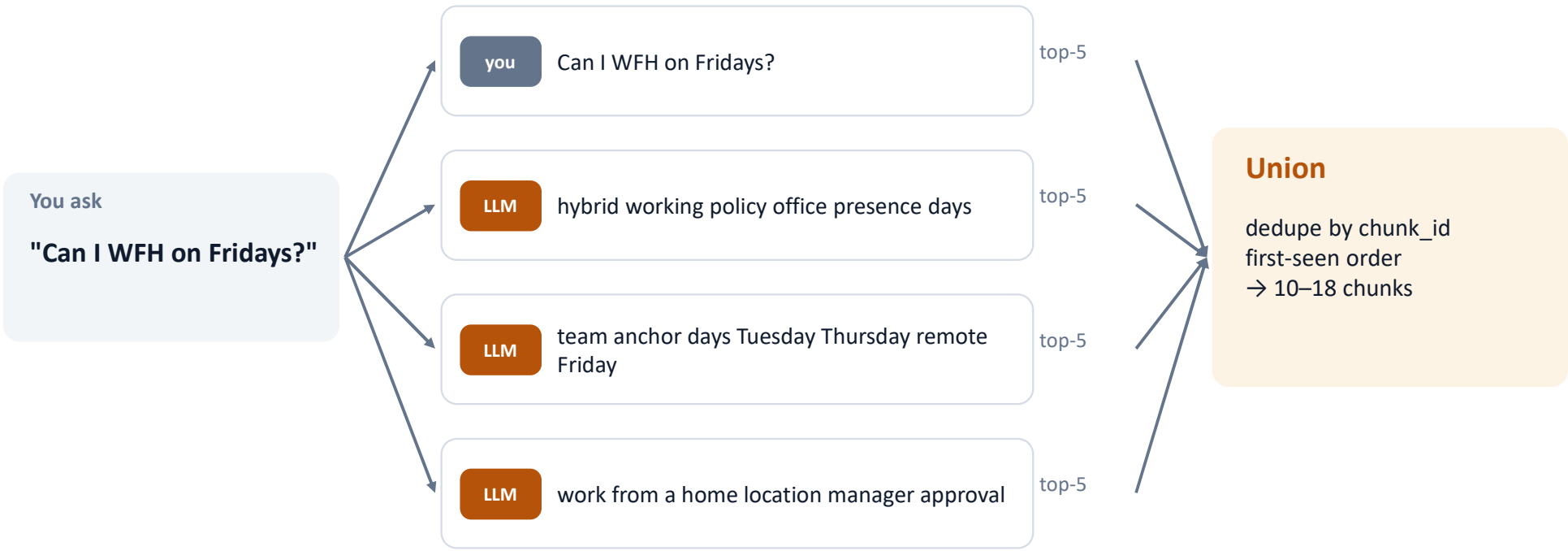
PART A · TODO 1–4 · app/mqr.py

Multi-Query Retriever

Users don't write good search queries. Let the LLM write three more, then search with all of them.

1. `parse_queries()`
2. `generate_queries()`
3. `unique_union()`
4. `mqr_retrieve()`

MQR: four nets in the water instead of one



Recall goes up: the 'anchor days' chunk is now found. **Price:** 2–3× more chunks in the prompt.

Skills in Part A: treat LLM output as untrusted input

1	`parse_queries()` **Defensive parsing**	Strip 1. / - / • / quotes, drop "Here are 3:" and duplicates, cap at n	tests 01–04
2	`generate_queries()` **Graceful degradation**	Blank input → no LLM call. Gateway error (429, timeout) → [] and carry on	tests 05–07
3	`unique_union()` **Identity by id**	Dedupe by metadata['chunk_id'], keep first occurrence and order	tests 08–09
4	`mqr_retrieve()` **Pipeline composition**	original + rewrites, top-MQR_K each, union, fallback flag	tests 10–12

Checkpoint A: tests 01–12 green · MQR mode answers the WFH question · inspector shows 4 queries

PART B · TODO 5–8 · app/rerank.py

Re-ranking with RRF

MQR found more. Now keep only the best five: rank the pool two ways and fuse the rankings.

5 `bm25_rank()`

6 `semantic_rank()`

7 `reciprocal_rank_fusion()`

8 `mqr_rrf_retrieve()`

Two ways to rank the same pool

Semantic rank

cosine(query vectors, chunk vector) · TODO 6

Good at

meaning, paraphrase: "WFH" ≈ "work from a home location"

Weak at

rare exact tokens: PAY-E4035 vs PAY-E4033

```
score = max over queries of
  dot(embed_query(q), vector_for(chunk))
```

BM25 rank

keyword relevance (rank_bm25.BM25Okapi) · TODO 5

Good at

exact words, codes, names: PAY-E4035, Ledgerline, KYC

Weak at

synonyms: "stole" vs "lost or stolen device" is fine, "WFH" vs "hybrid" is not

```
tokenize("PAY-E4035?")
  → ['pay-e4035', 'pay', 'e4035']
```

Reciprocal Rank Fusion: use positions, not scores

$$RRF(chunk) = \Sigma \; 1 / (k + rank)$$

sum over the rankings · rank starts at 1 · k = 60

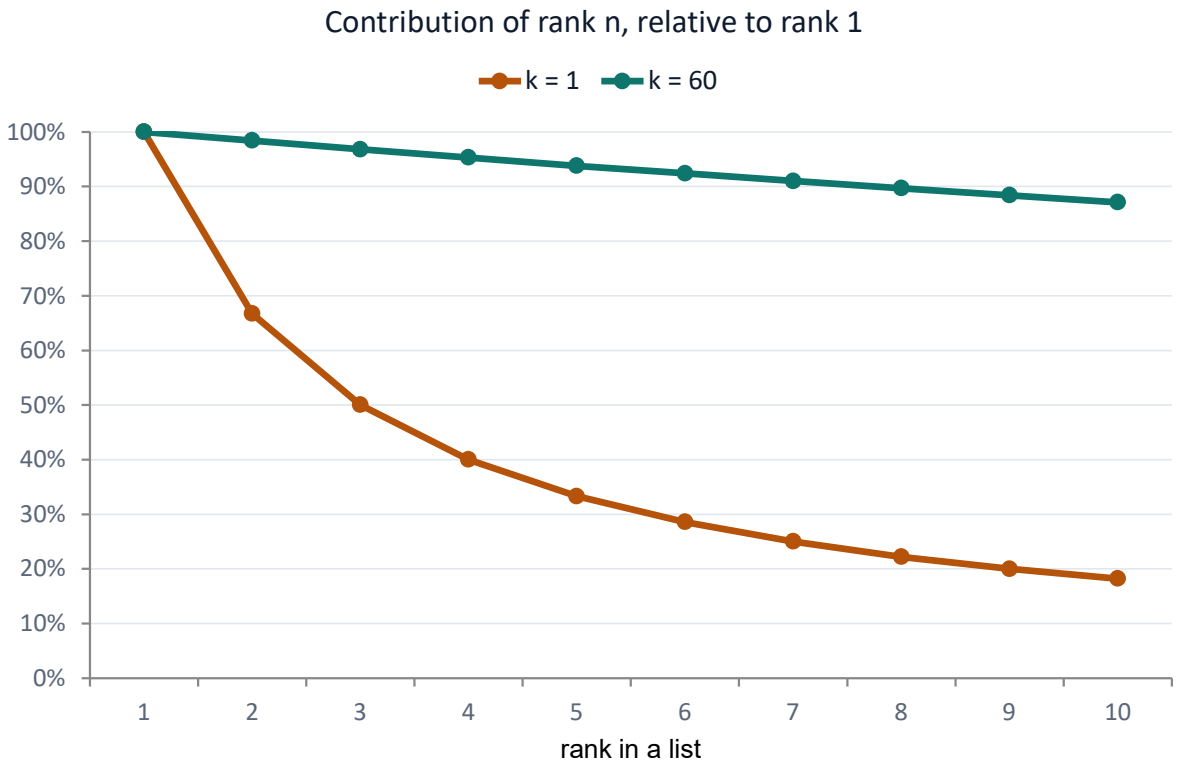
different score scales? no problem

- A chunk near the top of both lists wins
- Missing from a list: gets nothing from it
- Only the top FINAL_K = 5 go to the LLM

Chunk	Semantic	BM25	RRF score	Final
C1	#1	#2	`1/61 + 1/62 = 0.0325`	**1st**
C3	#3	#1	**`1/63 + 1/61 = 0.0323`**	**2nd**
C2	#2	–	`1/62 = 0.0161`	**3rd**
C4	–	#3	`1/63 = 0.0159`	**4th**

Read row C3: only 3rd by meaning, but 1st by keywords, so it beats C2, which is in one list only. This is how PAY-E4035 gets rescued.

What k = 60 does: no single list can dominate



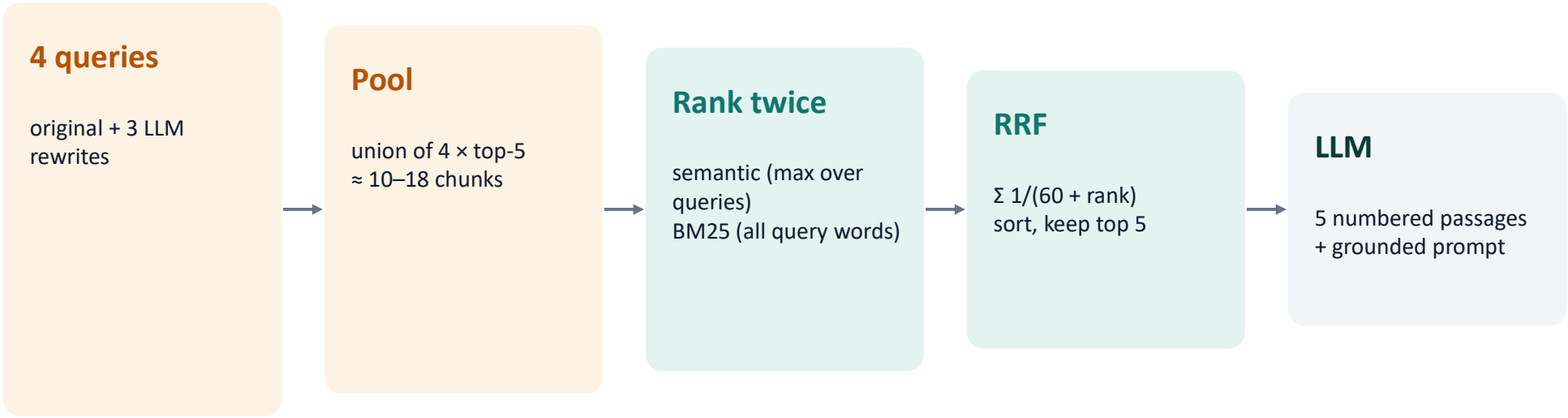
k = 1

Rank 2 is worth only 67% of rank 1. Whoever is first in any single list nearly wins.

k = 60 (default)

Rank 10 is still worth 87% of rank 1. Being good in both lists beats being first in one.

The full MQR + RRF pipeline (TODO 8)




```
mqr      = mqr_retrieve(store, question, cfg)
semantic = semantic_rank(store, mqr.queries, mqr.docs);  bm25 = bm25_rank(mqr.queries, mqr.docs)
fused    = reciprocal_rank_fusion([semantic, bm25], k=cfg.rrf_k, top_n=cfg.final_k)
```

Part C: better generation, not just better retrieval

Before: basic prompt

```
Answer the question using the context.

Context:
Tuesday and Thursday are team anchor
days ... Earned leave is credited ...
```

- No labels: model cannot cite
- Guesses when context is wrong
- All chunks, any order

After: grounded prompt

```
[1] (source: 02_hybrid_working.md)
Hybrid Working > Anchor days ...

[2] (source: 02_hybrid_working.md)
... cite like [1] or [2][3] ...
```

- TODO 9: numbered, best first, within MAX_CONTEXT_CHARS
- TODO 10: parse [1], [1][3], [1, 2]; drop invented [7]
- "I could not find this in the handbook." instead of guessing

Measure, don't guess: three numbers per mode

Hit rate

share of questions where the evidence reached the LLM

higher is better

MRR

average of 1 / position of the first correct chunk

1.0 = always first

Chunks sent

proxy for cost, latency and noise

lower is better at equal hit rate

MRR in 20 seconds

Q1	correct chunk at #1	`1/1 = 1.0`
Q2	correct chunk at #2	`1/2 = 0.5`
Q3	not found	`0`

`MRR = (1.0 + 0.5 + 0) / 3 = 0.5`

What teams usually see

- Basic: lowest hit rate on paraphrase and vague questions
- MQR: hit rate up, but 2–3× the chunks sent
- MQR + RRF: same hit rate as MQR, 5 chunks, best MRR

Think in edge cases: what the 25 tests check

Messy LLM output

numbering, quotes, preamble, duplicates, the question echoed back

tests 01–04

Service failure

429 / timeout → empty list, chat still works with the original question

tests 05–07, 11

Identity & order

dedupe by chunk_id, first occurrence wins, order kept

tests 08–10

Recall proof

Basic misses the WFH chunk, MQR finds it

tests 12

Ranking rules

1-based ranks, duplicates once, ties first-seen, top_n, k < 0

tests 13–20

Precision proof

≤ FINAL_K chunks; PAY-E4035 rescued by BM25 rank 1

tests 21–22

Budget & citations

skip oversize passages, keep the first; ignore [7] and [0]

tests 23–24

End to end

POST /api/chat in mqr_rrf mode, 422 on blank question

tests 25

What every fresher should be able to explain

1	**Why Basic RAG misses**	vocabulary mismatch, exact IDs, vague questions
2	**What MQR buys and costs**	recall up · 1 chat + 3 embed calls · 2–3× chunks
3	**Semantic vs BM25**	meaning vs exact words; why both
4	**RRF by hand**	1/(k + rank), worked example, role of k
5	**Grounded prompting**	numbered sources, budget, citations, "not found"
6	**Measuring retrieval**	hit rate, MRR, chunks sent

Common mistakes

- set(docs) → TypeError: use chunk_id
- 0-based ranks in RRF
- Forgetting the original query
- Sending the whole pool, not top-5
- Re-embedding in semantic_rank
- Trusting [7] when 5 passages exist
- Hard-coded model names or OpenAI() outside llm_client

WRAP-UP

Recall, then precision

MQR finds more. RRF keeps the best. Grounded prompts make answers checkable. Evaluation proves it.

Stretch goals

- RAG-Fusion: RRF over the 4 per-query lists
- Weighted RRF (semantic 1.0, BM25 0.5)
- Compare with LangChain MultiQueryRetriever + EnsembleRetriever
- HyDE: embed a hypothetical answer
- Add your own doc + 3 eval questions
- LLM re-ranker on top-10: gain vs cost